

UNIVERSIDAD DE GUADALAJARA

Universidad de Guadalajara Centro Universitario
de los Valles



Título: Manejo de errores y seguridad básica en flask

Materia: Desarrollo de bases de datos para web

Por: Heriberto Buenrostro Hernández

Docente: Adolfo Castillo Chavarín

Fecha de entrega: 06/11/2025

Contenido

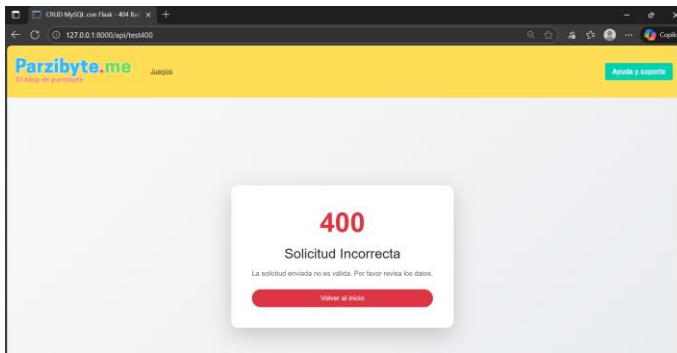
U1 - A5 Manejo de Errores y Seguridad Básica en Flask	3
Paginas de errores	3
Página 400 al usar abort(400).	3
Página 404 por ruta inexistente.	4
Página 405 por método no permitido.	4
Página 500 tras excepción no controlada.	5
Extracto de errors.log o interfaz de Sentry mostrando el error.	5
Código asignación de dns para la aplicación	5
Panel de control de sentry en este vienen todos los errores que se presentaron en la practica.....	6
Cómo registraste los manejadores y qué rutas probaste.	7
Resultados de las pruebas.	7
Reflexión: ¿Cómo mejora esto la seguridad y robustez de la aplicación?	7

U1 - A5 Manejo de Errores y Seguridad Básica en Flask

En esta práctica espero a como manejar errores que se le pueda presentar al usuario este tipo de herramientas de seguridad es clave para que el usuario para que tenga la mejor experiencia en la web, también espero implementarlo en mis futuros proyectos como desarrollador.

Páginas de errores

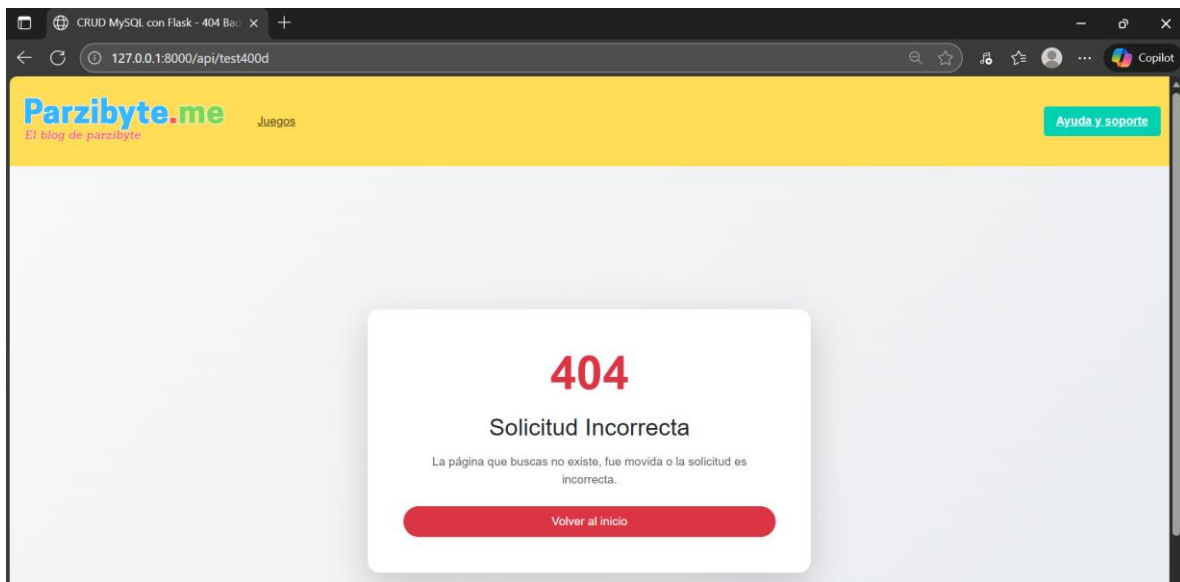
Página 400 al usar abort(400).



```
@app.errorhandler(BadRequest)
def error_400(e):
    return render_template('400.html'), 400
```

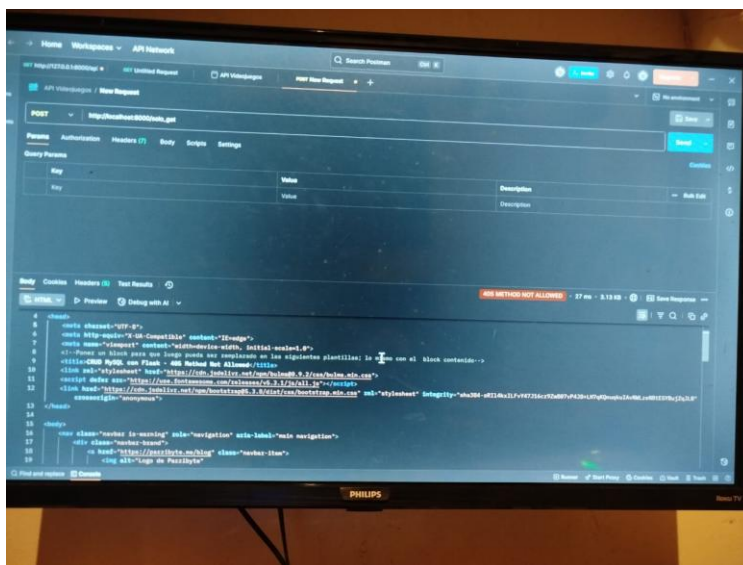
```
@app.route('/api/test400')
def test_400():
    abort(400, description="Solicitud incorrecta en el endpoint de prueba")
```

Página 404 por ruta inexistente.



```
3
4
5 @app.errorhandler(404)
6 def handle_404(e):
7     return render_template('404.html'), 404
```

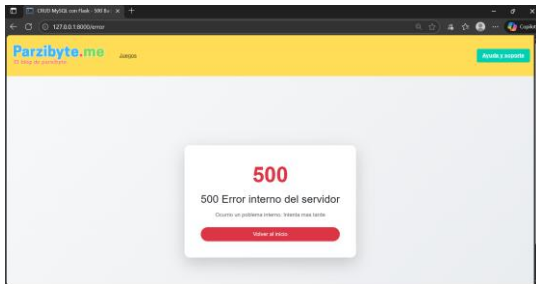
Página 405 por método no permitido.



```
3
4
5 @app.errorhandler(MethodNotAllowed)
6 def handle_405(e):
7     return render_template('405.html'), 405
```

```
@app.route('/solo_get', methods=['GET'])
def solo_get():
    return "Solo puedes usar GET aquí"
```

Página 500 tras excepción no controlada.



```
@app.errorhandler(500)
def internal_server_error(e):
    app.logger.error(f"Error interno del servidor: {e}")
    return render_template("500.html"), 500
```

```
@app.route("/error")
def error():
    1/0 #
    return "<p>error!</p>"
```

Extracto de errors.log o interfaz de Sentry mostrando el error.

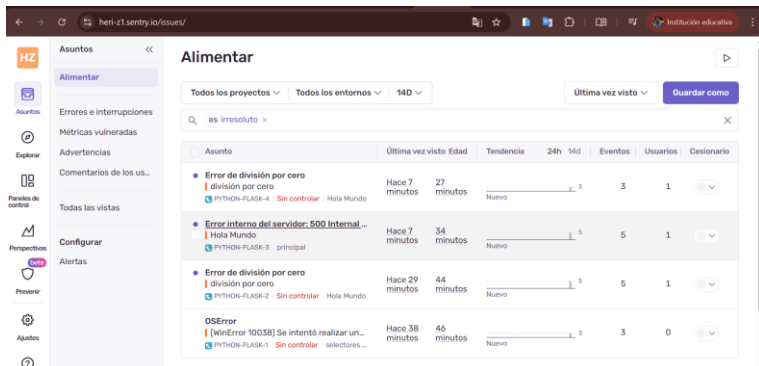
Código asignación de dns para la aplicación

```
import sentry_sdk
from sentry_sdk.integrations.flask import FlaskIntegration

app = Flask(__name__)

sentry_sdk.init(
    dsn="https://a6bcd545a463912f84e28d816232174@o4510321751556096.ingest.us.sentry.io/4510321757585408",
    # Add data like request headers and IP for users,
    # see https://docs.sentry.io/platforms/python/data-management/data-collected/ for more info
    send_default_pii=True,
)
```

Panel de control de sentry en este vienen todos los errores que se presentaron en la practica



The screenshot displays the Sentry dashboard for the 'Alimentar' project. The left sidebar contains navigation links for 'Asuntos', 'Alimentar', 'Errores e interrupciones', 'Métricas vulneradas', 'Advertencias', 'Comentarios de los us...', 'Todas las vistas', 'Configurar', 'Alertas', 'Proveer', and 'Ajustes'. The main content area shows a list of error reports with columns for 'Asunto', 'Última vez visto', 'Edad', 'Tendencia', '24h', '14d', 'Eventos', 'Usuarios', and 'Cuestionario'. The search bar indicates 'es irresoluto'.

Asunto	Última vez visto	Edad	Tendencia	24h	14d	Eventos	Usuarios	Cuestionario
• Error de división por cero división por cero PYTHON-FLASK-4 Sin controlar Hola Mundo	Hace 7 minutos	27 minutos	Nuevo	5	3	1		
• Error interno del servidor: 500 Internal... Hola Mundo PYTHON-FLASK-3 principal	Hace 7 minutos	34 minutos	Nuevo	5	5	1		
• Error de división por cero división por cero PYTHON-FLASK-2 Sin controlar Hola Mundo	Hace 29 minutos	44 minutos	Nuevo	5	5	1		
• OSError [winError 10038] Se intentó realizar un... PYTHON-FLASK-1 Sin controlar selectores...	Hace 38 minutos	46 minutos	Nuevo	5	3	0		

Cómo registraste los manejadores y qué rutas probaste.

En una aplicación Flask, los errores pueden clasificarse en errores HTTP (como 404 o 403) y estos errores no HTTP, son errores internos de Python, como una división por cero (error 500) un índice fuera de rango o el acceso a una variable inexistente. Cuando estos errores ocurren, Flask puede detener la ejecución y mostrar un mensaje de error en el navegador o, de forma más profesional, registrar el error en un archivo de logs por ejemplo a través de Sentry.

Los Fragmento de código que utilizamos no lo proporciona el microframework flask estos manejan errores http específicos. Lo primero que utilizamos son los decoradores que se utilizan para capturar algún error HTTP específico. Utilizamos este `@app.errorhandler(...)` para indicar que la función siguiente manejara una función luego en el decorador añadimos una clase en esta clase proviene del paquete Werkzeug. Que es la librería encargada del manejo de peticiones y errores HTTP, Luego se define la función controladora del error, el parámetro e contiene la información del error, por ejemplo: el mensaje y el tipo de excepción. Luego de que el decorador identifique el error a partir de un return lo va a direccionar a una página personalizada la cual tendrá un mensaje relacionado al error que se haya presentado.

Resultados de las pruebas.

Reflexión: ¿Cómo mejora esto la seguridad y robustez de la aplicación?

En esta práctica se implementó funciones para la seguridad de los usuarios estas funciones tomarían los errores estos los direccionarían por medios de rutas hacia páginas seguras para el usuario, asíendo que el usuario se sienta seguro en ellas, a través de las herramientas que ofrece flask. También aprendí una nueva herramienta llamada Sentry esta herramienta este me ayuda a identificar errores y me da un análisis de estos errores.